

Phát triển Chatbot và các hệ thống hỏi đáp với AI tạo sinh

Bài 3. Xây dựng chatbot với RASA

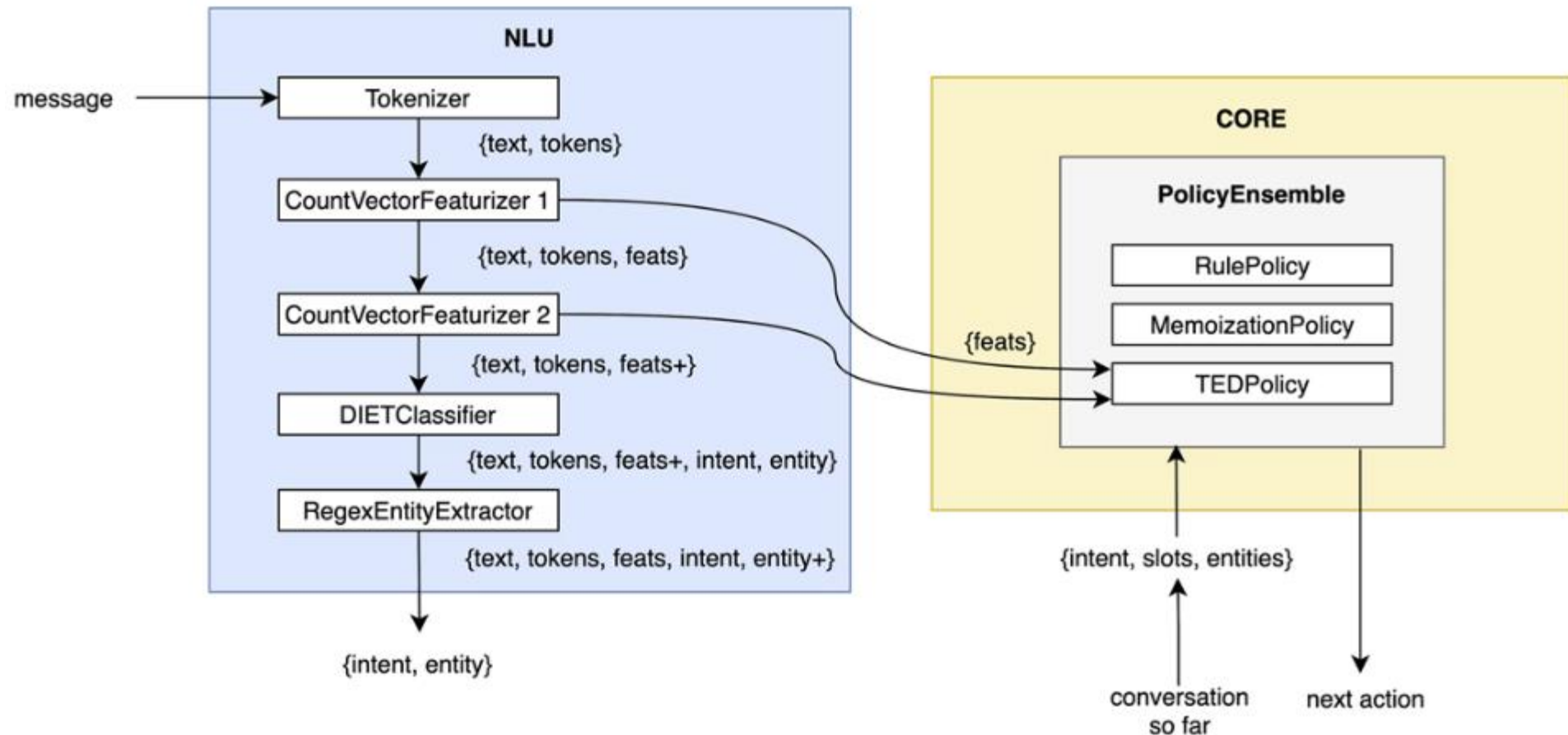
PGS.TS. Lê Thanh Hương
Trường Công nghệ thông tin và Truyền thông
Đại Học Bách Khoa Hà Nội

1. Khái niệm chung
2. Kiến trúc DIET
3. Huấn luyện DIET
4. So sánh DIET và BERT
5. Cách cải tiến DIET
6. Các bước tạo chatbot với RASA

1. Khái niệm chung

- **Rasa** là **framework mã nguồn mở** để phát triển các hệ thống hội thoại thông minh (Conversational AI), được thiết kế để **dễ tùy chỉnh, mở rộng và triển khai**
- **Một số tính năng nổi bật:**
 1. Huấn luyện chatbot bằng dữ liệu riêng
 2. Tùy chỉnh hoàn toàn, không phụ thuộc dịch vụ bên thứ ba
 3. Tích hợp dễ dàng với các nền tảng như Facebook, Telegram, Zalo, Web...
 4. Hỗ trợ xử lý ngôn ngữ tiếng Việt
 5. Kết hợp với machine learning hoặc rule-based
 6. Bảo mật tốt vì không gửi dữ liệu ra bên ngoài

Các thành phần của RASA (1)



Các thành phần của RASA (3)

1. RASA NLU (*Natural Language Understanding*)

Xử lý đầu vào câu hội thoại người dùng (tokenize, featurize), xác định ý định người dùng (Intent Classification) và trích chọn thực thể (Entity Extraction).

Các tính năng khác: Regular Expression, Synonyms, Lookup Table.

- **Regular Expression:** Trích chọn thực thể bằng Regex
VD: ngày tháng, từ khóa cụ thể
- **Synonym:** Từ đồng nghĩa
VD: "ub", "ubnd" và "Ủy ban nhân dân", khi gặp các từ "ub", "ubnd", hệ thống tự chuyển thành "Ủy ban nhân dân"
- **Lookup Table:** Định nghĩa một tập giá trị cho một biến (RASA gọi là slot)
VD: Slot Province, ta định nghĩa Lookup Table gồm 64 tên tỉnh thành: "Hà Nội", "Hải Phòng", "Hà Nam",... . Khi hệ thống phát hiện từ "Hải Phòng" trong câu hội thoại người dùng, hệ thống sẽ tự gán giá trị "Hải Phòng" cho biến Province trong bộ nhớ, điều này giúp chatbot có thể nhận ra một số giá trị bổ trợ cho ngữ cảnh

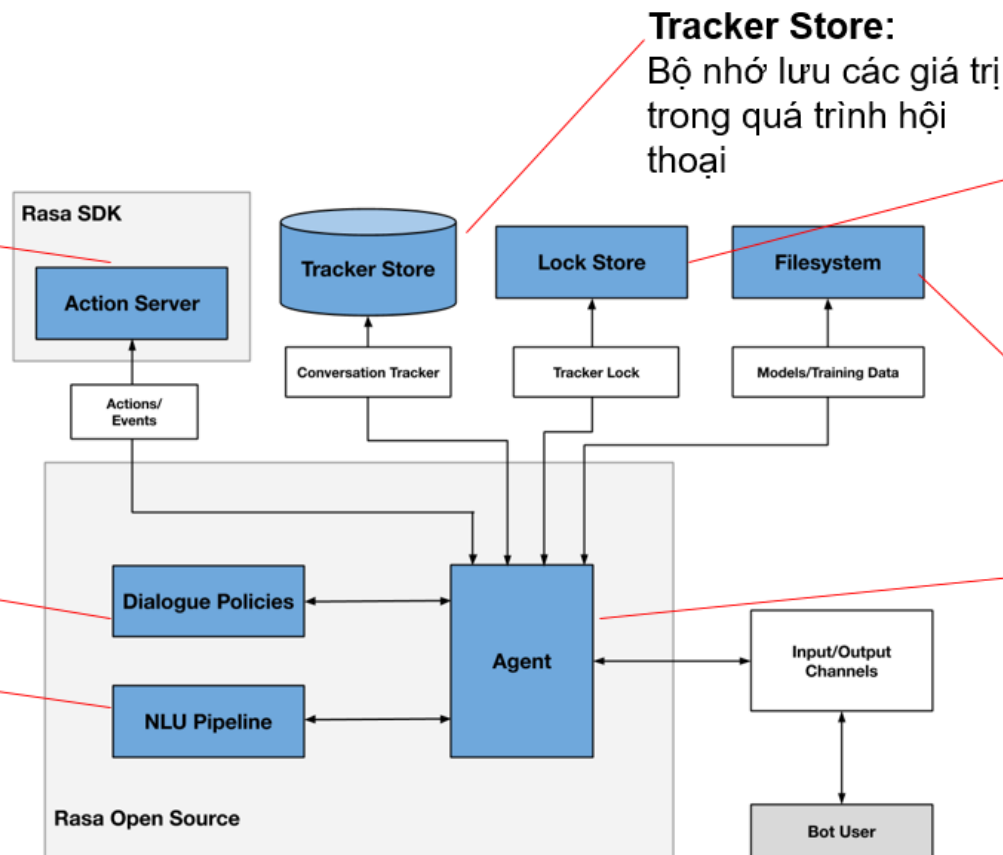
2. RASA Core

Kiểm soát luồng hội thoại, quyết định hành động nào được thực hiện để đáp lại câu hội thoại người dùng.

- Các chính sách để xử lý luồng hội thoại, gồm:
 - **RulePolicy**: Sử dụng các **luật** để xác định hành động tiếp theo
 - **MemoizationPolicy**: Sử dụng các **Story** để xác định hành động tiếp theo
 - **TEDPolicy**: Sử dụng **học sâu** để xác định hành động tiếp theo
- Các Policy này cùng nhau hợp lại thành Dialog Policies.

Action Server:
Các custom action
viết bằng Python

Hai thành phần chính:
Dialog Policies:
Điều hướng hội thoại
NLU Pipeline:
Xác định ý định người
dùng, trích xuất thực thể



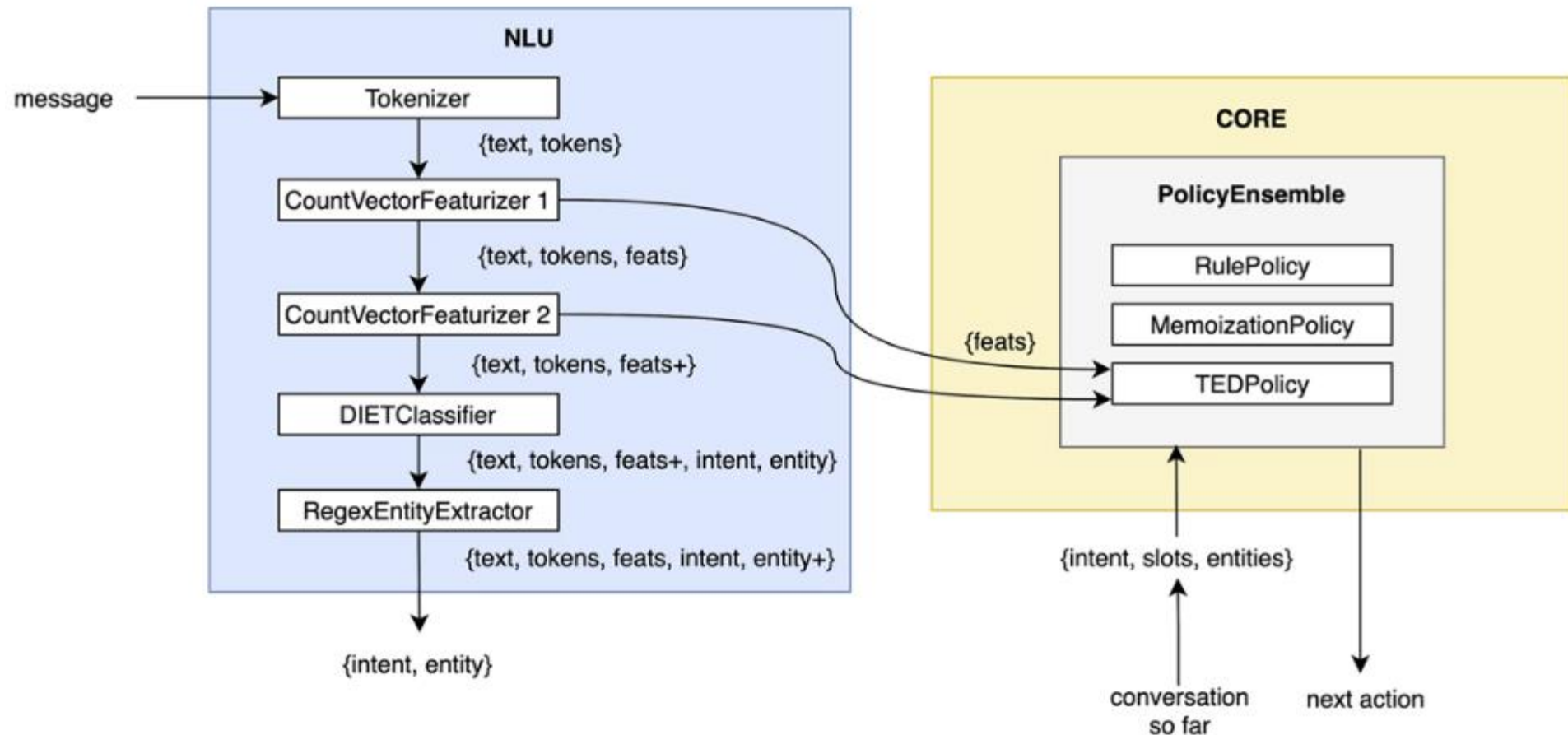
Tracker Store:
Bộ nhớ lưu các giá trị
trong quá trình hội
thoại

Lock Store:
Đảm bảo các message
được xử lý tuần tự,
không có race condition

File System:
Lưu trữ các file của
project chatbot

Agent:
Module xử lý chung.

Các thành phần của RASA (6)



1. Khái niệm chung

2. Kiến trúc DIET

3. Huấn luyện DIET

4. So sánh DIET và BERT

5. Cách cải tiến DIET

6. Các bước tạo chatbot với RASA

Kiến trúc DIET

Dual Intent and Entity Transformer (DIET) Classifier

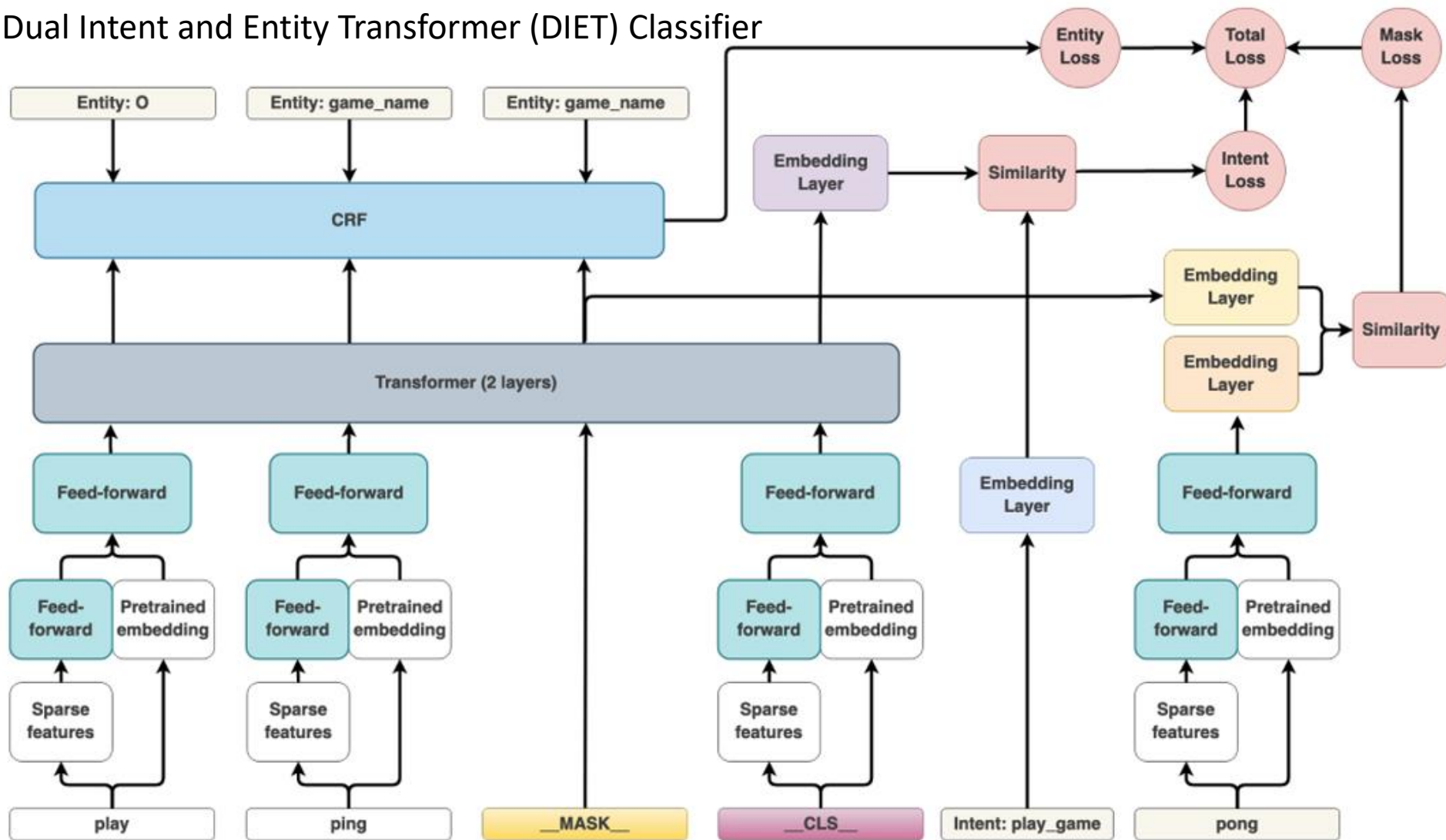
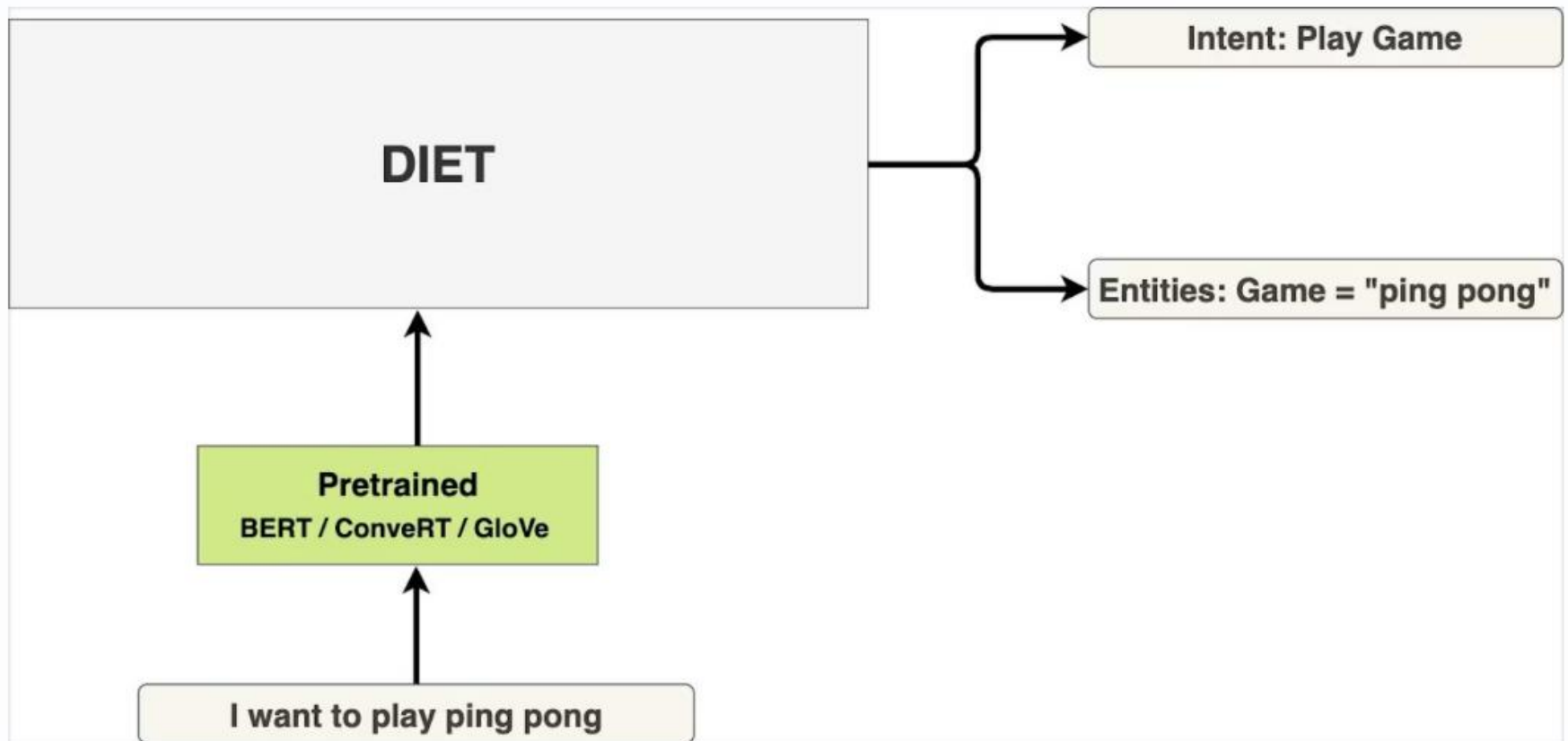


Figure 1: A schematic representation of the DIET architecture. The phrase "play ping pong" has the intent `play_game` and entity `game_name` with value "ping pong". Weights of the feed-forward layers are shared across tokens.

DIET Classifier

- Nhẹ, hiệu quả, cho kết quả vượt trội so với BERT, huấn luyện nhanh gấp 6 lần.



1. Khái niệm chung
2. Kiến trúc DIET
- 3. Huấn luyện DIET**
4. So sánh DIET và BERT
5. Cách cải tiến DIET
6. Các bước tạo chatbot với RASA

Huấn luyện DIET

- Thực hiện đồng thời ba tác vụ: dự đoán intent, trích chọn thực thể, và dự đoán masked token
- **Mask**: Một token trong câu được mask ngẫu nhiên. Quá trình huấn luyện phải đoán được từ này, **giúp khoảng cách các từ gần nghĩa gần nhau hơn** trên không gian ngữ nghĩa khi mô hình hội tụ.
- Hàm loss:

$$L_{total} = L_I + L_E + L_M$$

L_I , L_E , L_M lần lượt là hàm mất mát của việc xác định ý định, hàm mất mát của việc trích xuất thực thể, hàm mất mát của việc dự toán masked token

1. Khái niệm chung
2. Kiến trúc DIET
3. Huấn luyện DIET
- 4. So sánh DIET và BERT**
5. Cách cải tiến DIET
6. Các bước tạo chatbot với RASA

So sánh DIET và BERT (1)

DIET vượt trội hơn so với fine-tuned BERT và nhanh hơn gấp **6 lần** nhờ:

1. Kiến trúc Transformer hiệu quả

- DIET sử dụng mô hình dựa trên **Transformer nhẹ**, được tối ưu hóa cho **phân loại ý định và nhận diện thực thể**.
- Khác với BERT, DIET không xử lý toàn bộ vector ngữ cảnh mà tập trung vào việc học các biểu diễn **đặc thù cho tác vụ** một cách hiệu quả hơn.

2. Biểu diễn đặc trưng thưa

- DIET không phụ thuộc vào pre-trained embeddings như BERT mà sử dụng features như one-hot encodings mức token và multi-hot encodings của các n-grams mức ký tự.

"book" → trigrams = ["boo", "ook"]

→ vector = [1, 1, 0, 0, 0, ...]

- Giảm phụ thuộc vào các mô hình pre-trained lớn, nhưng vẫn đạt hiệu suất cao.

So sánh DIET và BERT (2)

3. Linh hoạt trong việc sử dụng pre-trained embedding vectors

- DIET cho phép tích hợp tùy chọn pre-trained embeddings vectors (BERT, GloVe, ConveRT), nhưng không bắt buộc phải sử dụng chúng.
- Ngay cả khi không sử dụng pre-trained embeddings vectors, DIET vẫn đạt hiệu suất tốt nhất.

4. Học tương phản (Contrastive Learning) để cải thiện biểu diễn

- Thay vì dựa vào phương pháp MLM như BERT, DIET sử dụng **contrastive loss function** để tối ưu hóa trực tiếp cho **phân loại ý định và nhận diện thực thể**.
- Việc tối ưu hóa **theo từng tác vụ cụ thể** này giúp mô hình đạt hiệu suất tốt hơn **với ít tham số hơn**.

5. Huấn luyện đa nhiệm đầu-cuối (End-to-End Multi-task Training)

- DIET tối ưu hóa **đồng thời** hai tác vụ **phân loại ý định** và **nhận diện thực thể**, giúp tăng hiệu quả huấn luyện.
- Trong khi đó, **BERT cần tinh chỉnh riêng** cho từng tác vụ, làm tăng độ phức tạp của quá trình huấn luyện.

6. Hiệu suất tính toán cao (Computational Efficiency)

- DIET **sử dụng ít tham số** và **tính toán hơn BERT**, giúp thời gian huấn luyện nhanh hơn đáng kể.
- Mô hình được thiết kế **để tối ưu tốc độ** mà **không làm giảm độ chính xác**, giúp nó phù hợp hơn cho triển khai thực tế.

1. Khái niệm chung
2. Kiến trúc DIET
3. Huấn luyện DIET
4. So sánh DIET và BERT
- 5. Cách cải tiến DIET**
6. Các bước tạo chatbot với RASA

Cách cải tiến DIET (1)

1. Cách huấn luyện DIET với dữ liệu mới

- Trộn dữ liệu huấn luyện ban đầu + dữ liệu mới để **huấn luyện tiếp**
- **Transfer learning** từ mô hình DIET đã được huấn luyện trước đó.
- Điều chỉnh **learning rate** phù hợp để tránh quên mất dữ liệu cũ

2. Sử dụng thêm pre-trained embeddings vectors

- Nếu mô hình hiện tại chỉ dùng **đặc trưng thưa** → thêm **word embeddings** để **tăng tính tổng quát**.
- Kết hợp các embedding vector từ **mô hình lớn hơn** để cải thiện khả năng **hiểu ngữ cảnh**.

Cách cải tiến DIET (2)

3. Điều chỉnh kiến trúc mô hình (Model Architecture Tuning)

- **Tăng số lớp Transformer hoặc số attention heads** để mô hình có khả năng học tốt hơn.
- **Sử dụng regularization** (như dropout) hợp lý để tránh overfitting khi mô hình lớn hơn.
- **Thay đổi activation function** hoặc thử nghiệm các kỹ thuật tối ưu hóa khác.

4. Sử dụng kỹ thuật huấn luyện nâng cao

- Sử dụng **contrastive learning** để cải thiện độ phân biệt giữa các ý định.
- **Multi-task Learning**: Nếu dữ liệu có liên quan đến nhiều tác vụ khác nhau, huấn luyện DIET đồng thời các dữ liệu đó để cải thiện hiệu suất.

1. Khái niệm chung
2. Kiến trúc DIET
3. Huấn luyện DIET
4. So sánh DIET và BERT
5. Cách cải tiến DIET
- 6. Các bước tạo chatbot với RASA**

Các bước tạo chatbot với RASA (1)

1. Cài Đặt Môi Trường

- Cài đặt Python (phiên bản 3.8, 3.9).
- Tạo môi trường ảo và cài đặt Rasa:

```
bash

# Cài đặt môi trường ảo (tùy chọn)
python -m venv rasa_env
source rasa_env/bin/activate # Linux/Mac
rasa_env\Scripts\activate    # Windows

# Cài đặt Rasa
pip install rasa
```

2. Khởi Tạo Rasa project

```
bash  
  
rasa init
```

Lệnh này sẽ tạo một thư mục chứa các file cần thiết để bắt đầu phát triển chatbot.

Các bước tạo chatbot với RASA (3)

3. Các tệp quan trọng:

- **domain.yml**: Định nghĩa intents, entities, responses, và actions. Định nghĩa logic phản hồi, nhưng không trực tiếp dùng để train mô hình. Giúp chatbot biết cách phản hồi khi hiểu được câu đó.
- **data/nlu.yml**: Chứa dữ liệu huấn luyện về các intent, giúp chatbot học cách hiểu câu của người dùng.
- **data/stories.yml**: Chứa kịch bản hội thoại
- **config.yml**: Chứa cấu hình pipeline xử lý NLP.
- **actions/actions.py**: Chứa các actions tùy chỉnh nếu cần.

Các bước tạo chatbot với RASA (4)

4. Tạo dữ liệu huấn luyện

File
domain.yml:
định nghĩa
intents, entities,
responses,
actions

```
intents:
  - greet
  - goodbye
  - ask_weather
entities:
  - location
slots:
  location:
    type: text
    influence_conversation: true
responses:
  utter_greet:
    - text: "Xin chào! Tôi có thể giúp gì cho bạn?"
  utter_goodbye:
    - text: "Tạm biệt! Hẹn gặp lại!"
  utter_ask_location:
    - text: "Bạn muốn biết thời tiết ở đâu?"
actions:
  - action_get_weather
```

Các bước tạo chatbot với RASA (5)

4. Tạo dữ liệu huấn luyện

- File `nlu.yml` : tạo dữ liệu huấn luyện
- Các từ nằm trong [] như [Hà Nội](location) là **entity** (location), giúp chatbot nhận diện vị trí trong câu.
- Khi người dùng nhập "Ở TP.HCM có mưa không?", chatbot sẽ biết `location = TP.HCM`.

```
nlu:  
- intent: greet  
  examples: |  
    - Xin chào  
    - Chào buổi sáng  
    - Rất vui được gặp bạn  
- intent: goodbye  
  examples: |  
    - Tạm biệt  
    - Hẹn gặp lại  
    - Bye  
    - Tạm biệt, chúc một ngày tốt lành  
- intent: ask_weather  
  examples: |  
    - Thời tiết hôm nay thế nào?  
    - [Hà Nội](location) hôm nay ra sao?  
    - Ở [TP.HCM](location) có mưa không?  
    - Thời tiết ở [Đà Nẵng](location) thế nào?  
    - Ngày mai [Hải Phòng](location) có lạnh không?
```

Các bước tạo chatbot với RASA (6)

4. Tạo dữ liệu huấn luyện

- File stories.yml: tạo kịch bản hội thoại

stories:

- story: Chào hỏi cơ bản

steps:

- intent: greet
- action: utter_greet

- story: Tạm biệt

steps:

- intent: goodbye
- action: utter_goodbye

- story: Hỏi thời tiết không có địa điểm

steps:

- intent: ask_weather
- action: utter_ask_location

- story: Hỏi thời tiết có địa điểm

steps:

- intent: ask_weather
- entities:
 - location: "Hà Nội"
- slot_was_set:
 - location: "Hà Nội"
- action: action_get_weather

Các bước tạo chatbot với RASA (7)

4. Tạo dữ liệu huấn luyện: File config.yml chứa cấu hình pipeline xử lý NLP

1 Cấu hình pipeline xử lý ngôn ngữ tự nhiên (NLU)

pipeline:

- name: **WhitespaceTokenizer** # Tách từ theo khoảng trắng (dùng cho tiếng Anh, tiếng Việt cần *Spacy*)
- name: **RegexFeaturizer** # Xử lý các pattern bằng regex (tốt cho số điện thoại, email, v.v.)
- name: **LexicalSyntacticFeaturizer** # Đưa ra đặc trưng dựa trên cú pháp của từ
- name: **CountVectorsFeaturizer** # Chuyển câu thành vector số để mô hình dễ học
- name: **DIETClassifier** # Mô hình học sâu nhận diện intent và entity
epochs: 100
- name: **EntitySynonymMapper** # Xử lý từ đồng nghĩa cho entity
- name: **ResponseSelector** # Chọn câu trả lời tốt nhất dựa trên câu hỏi
epochs: 100
- name: **FallbackClassifier** # Xử lý câu không hiểu rõ
threshold: 0.3
ambiguity_threshold: 0.1

2 Cấu hình chính sách hội thoại (Policies)

policies:

- name: **MemoizationPolicy** # Ghi nhớ các kịch bản hội thoại đã học
- name: **RulePolicy** # Áp dụng các quy tắc hội thoại (như fallback, FAQ)
- name: **TEDPolicy** # Mô hình học sâu giúp dự đoán bước tiếp theo trong hội thoại
epochs: 50



Các bước tạo chatbot với RASA (8)

Luồng hoạt động khi train hệ thống:

- Rasa đọc **config.yml** để biết sử dụng pipeline nào.
- Tải **data/nlu.yml** để huấn luyện mô hình nhận diện intent và entity.
- Tải **data/stories.yml** để huấn luyện mô hình hội thoại.
- Tải **domain.yml** để biết chatbot có những intent, response, action nào.
- Lưu mô hình đã train vào thư mục models/.

Luồng hoạt động khi chạy chatbot:

1. Người dùng nhập tin nhắn.
2. Rasa phân tích câu nói bằng pipeline trong config.yml để xác định intent và entity.
3. Kiểm tra domain.yml để xem intent đó có response hay action nào không.
4. Nếu intent yêu cầu thực hiện action, Rasa gọi actions/actions.py.
5. Trả lời người dùng bằng response từ domain.yml hoặc kết quả từ action.

Các bước tạo chatbot với RASA (9)

5. Huấn Luyện Mô Hình

```
bash  
  
rasa train
```

6. Chạy Chatbot

```
bash  
  
rasa shell
```

Các bước tạo chatbot với RASA (10)

7. Thêm Custom Action (Tuỳ Chỉnh Hành Động)

Nếu chatbot cần gọi API hoặc xử lý logic phức tạp, thêm vào actions.py:

```
python

from rasa_sdk import Action
from rasa_sdk.events import SlotSet

class ActionWeather(Action):
    def name(self):
        return "action_weather"

    def run(self, dispatcher, tracker, domain):
        location = tracker.get_slot("location")
        dispatcher.utter_message(text=f"Thời tiết tại {location} hiện tại là 30°C.")
        return []
```

Các bước tạo chatbot với RASA (11)

Cập nhật domain.yml:

```
yaml  
  
actions:  
  - action_weather
```

Chạy action server:

```
bash  
  
rasa run actions
```

8. Triển Khai Chatbot: Tích hợp chatbot với Telegram, Facebook Messenger hoặc tạo API REST để giao tiếp với ứng dụng web.